Version: 22092023

Advanced Python

Overview

Python is the fastest growing programming language because it has a strong community support. Many open-source Python libraries are ready for deployed for completing different tasks. With these powerful tools, programmers can save a lot of time without starting from scratch.

In this course, you will learn different Python libraries for data manipulation and visualization. At the end of the course, you will be capable of performing big data analysis and data visualization.

Introduction of Python Libraries

A Python library is a collection of pre-written code, functions, modules, and classes that can be used to perform various tasks and solve specific problems. Libraries in Python are designed to be reusable, making it easier for developers to work on their projects without having to reinvent the wheel by writing all the code from scratch.

```
#Example of using library with python
import math

number = 16
square_root = math.sqrt(number)

print(f“The square root is {square_root}”)
```

We need to import the library before using it

Output: The square root is 4

Unit 1

Datetime

What will you learn

- create datatime object with constructor
- perform arithmetic operation with datatime
- create a timezone-aware datetime
- represent datetime between applications

The datetime package

- Python can support date and time calculation: **import datetime**
- Represents a specific date and time: **datetime.datetime**
- Represents a duration of time: **datetime.timedelta**
- Represents a time zone offset from UTC: **datetime.timezone**

Create datetime object

- The datetime class can be created with its constructor: **datetime.datetime()**, for example:
  - datetime.datetime(1879,3,14)
- Using .now() to get the current local date and time: **datetime.datetime.now()**
- Using **.strptime()** to convert a string containing date and time information into a datetime object, for example:
  - **datetime.datetime.strptime**("1879-03-14", "%Y-%m-%d")

Datetime arithmetic

- The + and - operand is applicable to datetime, whenever make sense
- Difference between two datetime can be represented by timedelta:

  datetime - datetime -> timedelta
- Adding two datetime does not make any sense:

  datetime + datetime -> TypeError
- Adding timedelta to datetime:

  datetime +/- timedelta -> datetime
- Add/subtract two timedelta:

  timedelta +/- timedelta -> timedelta

Timezone

- We can create a timezone-aware datetime to represent a timezone defined by a fixed offset from UTC, for example:
  - timezone_vietnam = timezone(timedelta(hours=5, minutes=30))
- Before storing a log (a record with time) into a big data storage, it is common to convert the time to UTC time (+0) with timezone.utc and astimezone(), for example:
  - datetime.now(timezone_vietnam).**astimezone(timezone.utc)**

Datetime serialization and timestamp

- When it comes to communication between applications, we need some way to represent time. One option is to create a formatted time string with .strftime(), for example:
  - dt1 = datetime.now(timezone(timedelta(hours=8)))
  - dt1.**strftime**("%Y-%m-%dT%H:%M:%S%z") # A ISO 8601 format time string

- Another common format is the epoch timestamp. It represents the number of seconds since 1970-01-01 00:00, for example:
  - datetime.now(). **timestamp()**

Unit 2

Time

What will you learn

- use sleep function to pause the program for a certain duration

The time package

- We can use time.time() to return the time as a floating point number expressed in seconds since the epoch in UTC, for example:
  - import time

    epoch_timestamp = time.time()

- Sleep function can pause a program for a certain duration. It is useful in web scraping because it can give time for a webpage to load: **time.sleep**(s)

Unit 3

Random

What will you learn

- generate a random number between custom range
- pick a value from a list in random

Random

- Random functions have many use cases, such as shuffling, sample, and run simulations: **import random**
- .random() function in the random package generates a number from 0-1: **random.random()**
- .randint() function generates an integer with a range: **random.randint(),** for example:
  - random.randint(80, 90) generates a random integer in 80 ~ 90
- .choice() function pick a value from a list: **random.choice()**, for example:
  - random.choice(["husband", "wife"])

Unit 4

Introduction to Pandas

What will you learn

- Create a table from scratch
- Loading data from another file
- Selecting certain rows or columns of a table

Pandas's Data types

By far we have learnt some standard Python data types:

1. String
2. Integer
3. Float
4. Boolean
5. List
6. Dictionary
7. Set
8. Tuple
9. None Type

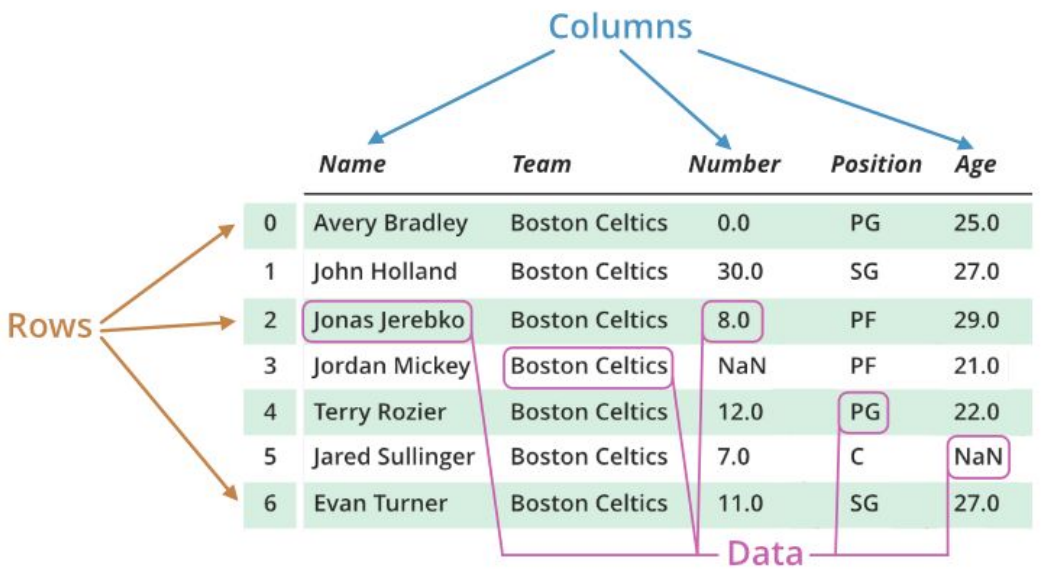
<u>New data types in Pandas</u>

1. DataFrame
2. Series
3. Index

Introduction of DataFrame/Series

A python DataFrame is a fundamental data structure provided by the popular data manipulation library called Pandas. A DataFrame can be thought of as a two-dimensional, tabular data structure, similar to a spreadsheet or a SQL table.

DataFrame



	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Series

	Name
0	Avery Bradley
1	John Holland
2	Jonas Jerebko
3	Jordan Mickey
4	Terry Rozier
5	Jared Sullinger
6	Evan Turner

How to create DataFrame

- We can create a pandas DataFrame in Python using either lists or dictionaries.

Lists

```
Import pandas as pd

df2 = pd.DataFrame ([
  ['John Smith', '123 Main St.', 34],
  ['Jane Doe', '456 Maple Ave.', 28],
  ['Joe Schmo', '789 Broadway', 51],
     ],
     columns=['name', 'address', 'age’]
     )
```

Dictionary

```
Import pandas as pd

df3 = pd.DataFrame([
{"name": 'John Smith', "address": '123 Main St.', 'age':
34},
{"name": 'Jane Doe', "address": '456 Maple Ave.’,
'age': 28},
{"name": 'Joe Schmo', "address": '789 Broadway’,
'age': 51},
])
```

Dateframe with CSV(Comma Separated Values)

- Most of the time, we'll be working with datasets that already exist.
- One of the most common formats for big datasets is the CSV.
- CSV (comma separated values) is a text-only spreadsheet format. The first row of a CSV contains column headings. All subsequent rows contain values. Each column heading and each variable is separated by a comma:

Example of CSV:

```
key1,key2,key3
value1,value2,value3
value4,value5,value6
```

Loading and Saving CSVs with Pandas Library

```
#Example of Loading and Saving CSVs
Import pandas as pd

df = pd.read_csv("new-csv-file.csv")

df.to_csv('new-csv-file.csv')
```

Inspect a DataFrame

- When loading a new DataFrame from a CSV, we want to know what it looks like.
- If it's a larger DataFrame, it's helpful to be able to inspect a few items without having to look at the entire DataFrame.

Example Dataframe: store_df

	Store_ID	Location	Number_of_Employees
0	1	San Diego	100
1	2	Los Angeles	120
2	3	San Diego	90
3	4	Sacramento	115

1.store_df.head(3)

	Store_ID	Location	Number_of_Employees
0	1	San Diego	100
1	2	Los Angeles	120
2	3	San Diego	90

2.store_df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Store_ID             4 non-null      int64
 1   Location             4 non-null      object
 2   Number_of_Employees  4 non-null      int64
dtypes: int64(2), object(1)
memory usage: 224.0+ bytes
```

3.store_df.describe()

	Store_ID	Number_of_Employees
count	4.000000	4.000000
mean	2.500000	106.250000
std	1.290994	13.768926
min	1.000000	90.000000
25%	1.750000	97.500000
50%	2.500000	107.500000
75%	3.250000	116.250000
max	4.000000	120.000000

Select column in DataFrame

- You can select column in a pandas DataFrame using various methods and techniques with Square Brackets [ ] as below:
  - Selecting single column : dataframe[column]
  - Selecting multiples columns : dataframe[[columns]]

Example dataframe: clinic_df

	month	east	north	south	west
0	Jan	100	100	23	100
1	Feb	51	45	145	45
2	Mar	81	96	65	96
3	Apr	80	80	54	180
4	May	51	54	54	154
5	Jun	112	109	79	129

Single column
clinic_df["east"]

	east
0	100
1	51
2	81
3	80
4	51
5	112

Multiple column
clinic_df[["month","east"]]

	month	east
0	Jan	100
1	Feb	51
2	Mar	81
3	Apr	80
4	May	51
5	Jun	112

Select row in DataFrame

- You can select row in a pandas DataFrame using various methods and techniques with below:
  - .iloc[ ] lookup a row based on the order of rows
  - .loc[ ] lookup a row based on index

Example dataframe: clinic_df

	month	east	north	south	west
0	Jan	100	100	23	100
1	Feb	51	45	145	45
2	Mar	81	96	65	96
3	Apr	80	80	54	180
4	May	51	54	54	154
5	Jun	112	109	79	129

Single row

clinic_df.loc[1] or clinic_df.iloc[1]

	month	east	north	south	west
1	Feb	51	45	145	45

Multiple rows

clinic_df.loc[2:3] or clinic_df.iloc[2:4]

	month	east	north	south	west
2	Mar	81	96	65	96
3	Apr	80	80	54	180

Pairwise operations

- Pandas Series provide easy pairwise math calculation.

  Try the following:
  - pd.Series + pd.Series
  - pd.Series - pd.Series
  - pd.Series * pd.Series
  - pd.Series / pd.Series
  - pd.Series ** pd.Series
  - pd.Series == pd.Series
  - pd.Series > pd.Series

Series

	number
0	1
1	2
2	3
3	4
4	5

pd.Series + pd.Series

pd.Series*2

Series

	number
0	2
1	4
2	6
3	8
4	10

Accessors

- When your series is of a specific data type, Pandas provides an interface to access dtype-specific methods, for example:
  - Datetime, Timedelta (.dt)
  - String (.str)
  - Some others, which we seldom concern

Series

	Word
0	"Abc"
1	”Bbc"
2	"BBc"
3	”AAC"
4	”DDD"

Series.**str**.lower()

Series

	Word
0	"abc"
1	”bbc"
2	"bbc"
3	”aac"
4	”ddd"

Filter Rows with boolean series

- You can select a subset of a DataFrame by using logical statements:
  df[df.MyColumnName == desired_column_value]

Example dataframe: clinic_df

	month	east	north	south	west
0	Jan	100	100	23	100
1	Feb	51	45	145	45
2	Mar	81	96	65	96
3	Apr	80	80	54	180
4	May	51	54	54	154
5	Jun	112	109	79	129

Single condition

clinic_df[df[“month”] == “Feb”]

	month	east	north	south	west
1	Feb	51	45	145	45

Filter Rows with boolean series

- You can select a subset of a DataFrame by using logical statements:
  df[df.MyColumnName == desired_column_value]

Example dataframe: clinic_df

	month	east	north	south	west
0	Jan	100	100	23	100
1	Feb	51	45	145	45
2	Mar	81	96	65	96
3	Apr	80	80	54	180
4	May	51	54	54	154
5	Jun	112	109	79	129

Multiple conditions

| for “or” , & for “and”, ~ for “not”

clinic_df[(clinic_df[“month”] == “Mar”) | (clinic_df[“north”] == “80”)]

	month	east	north	south	west
2	Mar	81	96	65	96
3	Apr	80	80	54	180

Isin with DataFrame (Boolean)

- You can select rows in a pandas DataFrame with isin like we do in list: df[MyColumnName].isin(list)

Example dataframe: clinic_df

	month	east	north	south	west
0	Jan	100	100	23	100
1	Feb	51	45	145	45
2	Mar	81	96	65	96
3	Apr	80	80	54	180
4	May	51	54	54	154
5	Jun	112	109	79	129

clinic_df[“month”].isin([“Jan”,”Feb”,”Mar”])

	month
0	True
1	True
2	True
3	False
4	False
5	False

Isin with DataFrame

- You can select rows in a pandas DataFrame with isin like we do in list:
  df [df[MyColumnName].isin(list)]

Example dataframe: clinic_df

	month	east	north	south	west
0	Jan	100	100	23	100
1	Feb	51	45	145	45
2	Mar	81	96	65	96
3	Apr	80	80	54	180
4	May	51	54	54	154
5	Jun	112	109	79	129

clinic_df[clinic_df[“month”].isin([“Jan”,”Feb”,”Mar”])]

	month
0	Jan
1	Feb
2	Mar

Unit 5

Transform Tables With Pandas

What will you learn

- renaming columns and index
- inplace operation
- updating columns and index
- applying functions on tables

Renaming columns and index

- **Changing column names directly** - column names can also be altered by directly replace the variable in .columns attribute of a dataframe.

Example dataframe: clinic_df

	month	east	north	south	west
0	Jan	100	100	23	100
1	Feb	51	45	145	45
2	Mar	81	96	65	96
3	Apr	80	80	54	180
4	May	51	54	54	154
5	Jun	112	109	79	129

```
df_copy = clinic_df.copy()
df_copy.columns = ["Col1", "Col2", "Col3", "Col4", "Col5"]
```

	Col1	Col2	Col3	Col4	Col5
0	Jan	100	100	23	100
1	Feb	51	45	145	45
2	Mar	81	96	65	96
3	Apr	80	80	54	180
4	May	51	54	54	154
5	Jun	112	109	79	129

Renaming columns and index

- **Using .rename()** - we can rename both column names and index (row name) with the rename function. A dictionary is required as parameter.

Example dataframe: clinic_df

	month	east	north	south	west
0	Jan	100	100	23	100
1	Feb	51	45	145	45
2	Mar	81	96	65	96
3	Apr	80	80	54	180
4	May	51	54	54	154
5	Jun	112	109	79	129

clinic_df.rename(columns={"month": "Month", "east": "East", "north": "NORTH"},inplace=True)

	Month	East	North	south	west
0	Jan	100	100	23	100
1	Feb	51	45	145	45
2	Mar	81	96	65	96
3	Apr	80	80	54	180
4	May	51	54	54	154
5	Jun	112	109	79	129

Inplace vs non-inplace operations

- **Inplace** means the operation will modify the original object.
- **Non-Inplace** means the operation will not modify the original object, a new object will be returned.

Inplace

```
df['col1'] = 1

df.columns = ["col1", "col2", "col3"]

df.index = ["Peter", "Mary", "Jude"]
```

Non-Inplace

```
df.reset_index()

df.rename()
```

The benefit for non-inplace function:

- we are able to chain up functions like df.rename().reset_index().groupby().....
- The original object is not changed, so that we can redo commands during analysis

Adding a column

- There are various ways to add column to the dataframe

Assign to key	.insert()	.assign()
df[“new_column”]=[1,2,3,4] df[“new_column”]= df[“column”] +1	df.insert(0,”new_column”,[1,2,3,4])	df.assign(new_column=[1,2,3,4])

Index

- You can set the index of a dataframe or series with
  - Column name
  - Series
- Note that .set_index() **DOES NOT mutate** original dataframe.

set_index()

	First Name	Last Name
100	John	Smith
200	Jane	Doe
300	Joe	Schmo

reset_index()

	index	First Name	Last Name
0	**0**	John	Smith
1	**1**	Jane	Doe
2	**2**	Joe	Schmo

Multi index

		marks
student_id	**exam_id**	
1	1	90
	2	86
2	1	95
	2	93

Adding rows

- We can add a single row using df.loc.
- We can get the number of rows using len(df.index) for determining the position at which we need to add the new row
- To add multiple rows, we can use pd.concat

```
df1 = pd.DataFrame([
    [1, 1, 90],
    [1, 2, 86],
    [2, 1, 95],
    [2, 2, 93],
   ],
   columns=["student_id", "exam_id", "marks"]
)

# since len(df.index()) is 4, it will add a new row
with index 4 and value [3, 1, 100]

df1.loc[len(df.index)] = [3, 1, 100]
```

```
df2 = pd.DataFrame([
    [3, 1, 95],
    [3, 2, 90],
    [4, 1, 85],
    [4, 2, 80],
   ],
   columns=["student_id", "exam_id", "marks"]
)

# append rows in df2 to df1 with index reset

df3 = pd.concat([df1, df2], ignore_index = True)
```

Apply function in dataframe/series

- In Pandas, we could also apply function in series and dataframe with **. apply()**
- Series' apply: Take 1 input, send 1 output
- DataFrame's apply: Take 1 series of input, send 1 output
- Can operate in column mode (axis=0) or row mode (axis=1)
- Row mode is more common is data analytics
- To access particular values of the row, we use the syntax row.column_name or row[‘column_name’].

Apply function in dataframe/series

```
def create_email_heading(row):
# row is a series
# we can use row['Name'] to access the value because 'Name' is one of the index value of row
return row['Name'] + "<" + row['Email'] + ">"

email_df["Email heading"] = email_df.apply(create_email_heading, axis=1)
```

email_df

	Name	Email
0	Jennie Kim	jennie@gmail.com
1	Jisoo	jisoo@gmail.com
2	Rose	rose@hotmail.com
3	Lalisa	lalisa@yahoo.com

	Name	Email	provider	Email heading
0	Jennie Kim	jennie@gmail.com	gmail.com	Jennie Kim<jennie@gmail.com>
1	Jisoo	jisoo@gmail.com	gmail.com	Jisoo<jisoo@gmail.com>
2	Rose	rose@hotmail.com	hotmail.com	Rose<rose@hotmail.com>
3	Lalisa	lalisa@yahoo.com	yahoo.com	Lalisa<lalisa@yahoo.com>

Short hand:lambda

- We can use lambda function as short hand.
- lambda *arguments* : *expression*

```
email_df['provider'] = email_df['Email'].apply(lambda n : n.split('@')[-1])
```

email_df

	Name	Email
0	Jennie Kim	jennie@gmail.com
1	Jisoo	jisoo@gmail.com
2	Rose	rose@hotmail.com
3	Lalisa	lalisa@yahoo.com

	Name	Email	provider
0	Jennie Kim	jennie@gmail.com	gmail.com
1	Jisoo	jisoo@gmail.com	gmail.com
2	Rose	rose@hotmail.com	hotmail.com
3	Lalisa	lalisa@yahoo.com	yahoo.com

Unit 6

Aggregate Operation With Pandas

What will you learn

- sorting the values in the table
- apply aggregate functions including count, sum, mean
- establish data groups based on columns and aggregates the information within the group with Group by functions
- transform the tables with transpose, pivot and melt

Sorting

- We can reorder the rows (in a df) or values(in a series) with .sort_values():
  - df.sort_values('column_name')
  - df.sort_values(['column_name1', 'column_name2'])

store_df.sort_values('['Date','Weekly_Sales']).head()

	Store	Dept	Date	Weekly_Sales	IsHoliday
0	1	1	05/02/2010	24924.50	False
1	1	1	12/02/2010	46039.49	True
2	1	1	19/02/2010	41595.55	False
3	1	1	26/02/2010	19403.54	False
4	1	1	05/03/2010	21827.90	False

	Store	Dept	Date	Weekly_Sales	IsHoliday
203398	21	47	01/04/2011	-399.0	False
222781	23	47	01/04/2011	-206.0	False
113827	12	47	01/04/2011	-55.0	False
387356	41	54	01/04/2011	-53.2	False
336527	35	49	01/04/2011	-28.0	False

Sorting - sort_index()

- It is also possible to sort by the index: **df.sort_index**(ascending=False)

store_df.sort_index(ascending=False).head()

	Store	Dept	Date	Weekly_Sales	IsHoliday
0	1	1	05/02/2010	24924.50	False
1	1	1	12/02/2010	46039.49	True
2	1	1	19/02/2010	41595.55	False
3	1	1	26/02/2010	19403.54	False
4	1	1	05/03/2010	21827.90	False

	Store	Dept	Date	Weekly_Sales	IsHoliday
421569	45	98	26/10/2012	1076.80	False
421568	45	98	19/10/2012	760.01	False
421567	45	98	12/10/2012	1061.02	False
421566	45	98	05/10/2012	628.10	False
421565	45	98	28/09/2012	508.37	False

Aggregate Function

- There are a lot of aggregation functions. Here's some of the common ones:
  - **1.series.count()**
    Returns the number of non-null (non-missing) values in the Series
  - **2.series.sum()**
    Calculates the sum of all the values in the Series
  - **3.series.mean()**
    Computes the average (mean) of the values in the Series
    4.series.max()
    Finds the maximum (largest) value in the Series
  - **5.series.min()**
    Finds the minimum (smallest) value in the Series
  - **6.series.unique()**
    Returns an array of unique values in the Series, removing duplicates
  - **7.series.nunique()**
    Counts the number of unique values in the Series

Aggregate Function examples

	Courses	Fee	Duration	Discount
0	Spark	20000	30	1000
1	PySpark	25000	40	2300
2	Hadoop	26000	35	1200
3	Python	22000	40	2500
4	PySpark	24000	60	2000
5	Spark	35000	60	2000

1.df[‘Fee’].count() □ 6

2. df[‘Fee’]. sum() □ 152000

3. df[‘Fee’]. mean() □ 25333.33

4. df[‘Fee’]. max() □ 35000

5. df[‘Fee’].min() □ 20000

6. df[‘Fee’]. unique() □

array([20000, 25000, 26000, 22000, 24000, 35000])

7. df[‘Fee’].nunique() □ 6

Group by - aggregate pattern

- Group by is an essential feature in every table manipulation. In pandas, we use the function: **df.groupby**("Column name")

- After a group by, we must provide further aggregation instruction so that we can obtain usable information, for example:
  - df.groupby("Column name").sum()
  - df.groupby("Column name").max()
  - df.groupby("Column name").agg(func)
  - df.groupby("Column name").agg([func1, func2])
  - df.groupby("Column name").agg({"col1": func1, "col2": func2})

The GroupBy object

- After a .groupby() function is run, a DataFrameGroupBy object is produced.
- A DataFrame GroupBy object represents a intermediate result where records are grouped but not aggregated yet.
- In this stage, we may further select the columns before proceed to aggregation.
  - [['col1', 'col2]] returns a DataFrameGroupBy
  - ['col'] syntax returns a SeriesGroupBy object
- In the following example, you can expect the result to be a DataFrameGroupBy object
  - **store_df.groupby(['Store','Dept'])[['Weekly_Sales', 'Date']]**

The special count() aggregation

- Count is different from other aggregation method like sum and max because we do not need numbers to be calculated.

```
df1.groupby('Courses').count()
```

	Courses	Fee	Duration	Discount
0	Spark	20000	30	1000
1	PySpark	25000	40	2300
2	Hadoop	26000	35	1200
3	Python	22000	40	2500
4	PySpark	24000	60	2000
5	Spark	35000	60	2000

	Fee	Duration	Discount
Courses			
Hadoop	1	1	1
PySpark	2	2	2
Python	1	1	1
Spark	2	2	2

The .value_counts() function

- However, the result is weird since the count() aggregation method is applied on all the three columns Fee, Duration and Discount. Turns out the correct way is to use .value_counts.

```
df1.groupby('Courses') .value_counts()
```

	Courses	Fee	Duration	Discount
0	Spark	20000	30	1000
1	PySpark	25000	40	2300
2	Hadoop	26000	35	1200
3	Python	22000	40	2500
4	PySpark	24000	60	2000
5	Spark	35000	60	2000

```
Spark 2
PySpark 2
Hadoop 1
Python 1
Name: Courses, dtype: int64
```

Time series Group by

- Time series is a series that represent measures in regular time interval.
- Examples include:
  - Per minute sleep quality
  - Hourly temperature
  - Daily sales volume

	time	reading
0	1988-04-07 00:00	30
1	1988-04-07 12:00	29
2	1988-04-08 00:00	30
3	1988-04-09 00:00	29
4	1988-04-09 12:00	28
5	1988-04-10 03:00	31

.resample()

- While modeling time series can be very valuable, we must first preprocess them such that the interval between records are fixed.
- **.resample()** is a way to group by a time interval (e.g. 1 day)

	time	reading
0	1988-04-07 00:00	30
1	1988-04-07 12:00	29
2	1988-04-08 00:00	30
3	1988-04-09 00:00	29
4	1988-04-09 12:00	28
5	1988-04-10 03:00	31

```
df.index = pd.to_datetime( df['time'],
format="%Y-%m-%d %H:%M")
```

```
time
1988-04-07 00:00:00 30
1988-04-07 12:00:00 29
1988-04-08 00:00:00 30
1988-04-09 00:00:00 29
1988-04-09 12:00:00 28
1988-04-10 03:00:00 31
Name: reading, dtype: int64
```

```
df.resample("1d")['reading'].mean()
```

```
time
1988-04-07 29.5
1988-04-08 30.0
1988-04-09 28.5
1988-04-10 31.0
Freq: D,
Name: reading,
dtype: float64
```

.rolling()

- The analytic methods "moving average" or "rolling sum" are achieved with the **.rolling()** method, for example:
  - We wish to generate a 2-days moving average on the previous dataframe.
  - Note: this method does not require the index to be a datetime index.

```
df.resample("1d")['reading'].mean()
```

```
time
1988-04-07 29.5
1988-04-08 30.0
1988-04-09 28.5
1988-04-10 31.0
Freq: D,
Name: reading,
dtype: float64
```

```
df.rolling(2).mean()
```

```
time
1988-04-07 NaN
1988-04-08 29.75
1988-04-09 29.25
1988-04-10 29.75
Freq: D,
Name: reading,
dtype: float64
```

Transpose, Pivot and Melt

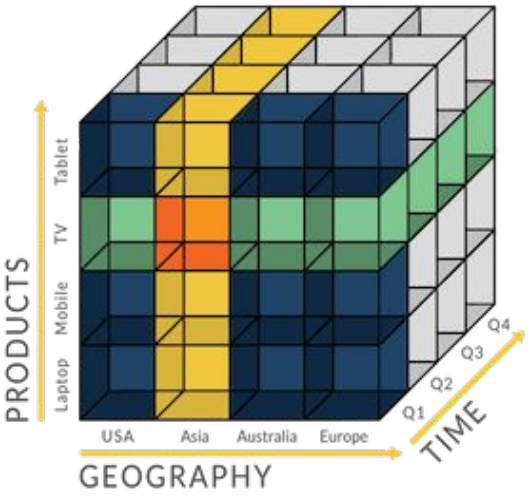
- Transpose is a mathematical operation on a matrix.
- The operation is to flip columns and rows.
- It is done by using the **dataframe.T** operation.

	0	1	2	3
0	1	1	3	4
1	2	2	5	8

df.T

	0	1
0	1	2
1	1	2
2	3	5
3	4	8

- In the following example, the dataframe is a "flatted", "melted" or "unpivoted" state.
- There is a name to describe it as "OLAP cube". This is the most common form of data in big data storage.



pd.pivot_table()

- Pivot is to transform the flattened form into a 2d grid.
- The **pd.pivot_table(df, index="",columns="",values="", aggfunc="")** is the general form of pivoting that involves 2 steps
  - Aggregation
  - Rearrange values

	Store	Dept	Date	Weekly_Sales	IsHoliday
0	1	1	05/02/2010	24924.50	False
1	1	1	12/02/2010	46039.49	True
2	1	1	19/02/2010	41595.55	False
3	1	1	26/02/2010	19403.54	False
4	1	1	05/03/2010	21827.90	False

Store	1	2	3	4	5
Dept					
1	3219405.18	4401251.25	1047992.81	5288131.43	1397761.09
2	6592598.93	9425547.92	2408373.92	13390422.10	1761467.32
3	1880518.36	2499148.56	787830.01	2718786.31	586455.18
4	5285874.09	6521896.32	1206088.67	8094286.22	1410095.37
5	3468885.58	4369410.09	1672437.42	6530582.17	957985.92

DataFrame.pivot()

- If we do not need any aggregation during the pivoting, we can use the **df.pivot(index=””, columns=””, values=””)** function directly.

	Store	Dept	Total_Sales
0	1	1	3219405.18
1	1	2	6592598.93
2	1	3	1880518.36
3	1	4	5285874.09
4	1	5	3468885.58

Store	1	2	3	4	5
Dept					
1	3219405.18	4401251.25	1047992.81	5288131.43	1397761.09
2	6592598.93	9425547.92	2408373.92	13390422.10	1761467.32
3	1880518.36	2499148.56	787830.01	2718786.31	586455.18
4	5285874.09	6521896.32	1206088.67	8094286.22	1410095.37
5	3468885.58	4369410.09	1672437.42	6530582.17	957985.92
...	...	...	...	...	...
95	17270404.89	20533191.52	2251610.54	21054815.74	2765719.24
96	4056723.46	4907626.13	562639.22	5483559.95	855950.97
97	5034650.88	5819700.18	48081.23	5625511.17	84717.93
98	1691371.22	2007062.32	1100.55	2146322.66	689.45
99	11325.37	19987.67	NaN	26173.66	NaN

81 rows × 45 columns

series.unstack()

- There's a chance that we have a multi-index series that we want to pivot directly
- We will use **series.unstack()** to create the pivot table with the multi-index:

```
pivoted = aggregated_srs.unstack()
```

Dept	Store	Weekly sales
1	1	3219405.18
2	2	4401251.25
3	3	1047992.81
4	4	5288131.43
5	5	1397761.09
6	6	13729.66
7	7	6025.47
8	8	6025.47
9	9	6025.47
10	10	19080.67

Store	1	2	3	4	5	6	7	8
Dept								
1	3219405.18	4401251.25	1047992.81	5288131.43	1397761.09	3413060.19	1364620.58	2114945.31
2	6592598.93	9425547.92	2408373.92	13390422.10	1761467.32	7188529.53	3232327.78	5109364.51
3	1880518.36	2499148.56	787830.01	2718786.31	586455.18	2403349.35	1234595.78	1527712.63
4	5285874.09	6521896.32	1206088.67	8094286.22	1410095.37	4888793.41	2137924.16	3015771.23
5	3468885.58	4369410.09	1672437.42	6530582.17	957985.92	4928538.99	1982030.12	2836955.44
...	...	...	...	...	...	...	...	...
95	17270404.89	20533191.52	2251610.54	21054815.74	2765719.24	12756856.44	4891757.91	9002059.31
96	4056723.46	4907626.13	562639.22	5483559.95	855950.97	4354427.54	160643.80	50.96
97	5034650.88	5819700.18	48081.23	5625511.17	84717.93	2951186.39	625614.69	2427906.41
98	1691371.22	2007062.32	1100.55	2146322.66	689.45	1391118.39	36785.01	983906.70
99	11325.37	19987.67	NaN	26173.66	NaN	15545.47	15.00	10435.38

81 rows × 45 columns

Reverse the pivot process -- .stack()

- **.stack()** can be use to reverse the pivot process.

Store	1	2	3	4	5	6	7	8
Dept								
1	3219405.18	4401251.25	1047992.81	5288131.43	1397761.09	3413060.19	1364620.58	2114945.31
2	6592598.93	9425547.92	2408373.92	13390422.10	1761467.32	7188529.53	3232327.78	5109364.51
3	1880518.36	2499148.56	787830.01	2718786.31	586455.18	2403349.35	1234595.78	1527712.63
4	5285874.09	6521896.32	1206088.67	8094286.22	1410095.37	4888793.41	2137924.16	3015771.23
5	3468885.58	4369410.09	1672437.42	6530582.17	957985.92	4928538.99	1982030.12	2836955.44
...	...	...	...	...	...	...	...	...
95	17270404.89	20533191.52	2251610.54	21054815.74	2765719.24	12756856.44	4891757.91	9002059.31
96	4056723.46	4907626.13	562639.22	5483559.95	855950.97	4354427.54	160643.80	50.96
97	5034650.88	5819700.18	48081.23	5625511.17	84717.93	2951186.39	625614.69	2427906.41
98	1691371.22	2007062.32	1100.55	2146322.66	689.45	1391118.39	36785.01	983906.70
99	11325.37	19987.67	NaN	26173.66	NaN	15545.47	15.00	10435.38

81 rows × 45 columns

Dept	Store	Weekly sales
1	1	3219405.18
2	2	4401251.25
3	3	1047992.81
4	4	5288131.43
5	5	1397761.09
6	6	13729.66
7	7	6025.47
8	8	6025.47
9	9	6025.47
10	10	19080.67

Reverse the pivot process -- .melt()

- If the dimension columns is not the index, we may use **.melt()** instead, for example:

```
dept_is_in_a_column.melt(id_vars="Dept")
```

Store	Dept	1	2	3	4
0	1	3219405.18	4401251.25	1047992.81	5288131.43
1	2	6592598.93	9425547.92	2408373.92	13390422.10
2	3	1880518.36	2499148.56	787830.01	2718786.31
3	4	5285874.09	6521896.32	1206088.67	8094286.22
4	5	3468885.58	4369410.09	1672437.42	6530582.17
...	...	...	...	...	...
76	95	17270404.89	20533191.52	2251610.54	21054815.74
77	96	4056723.46	4907626.13	562639.22	5483559.95
78	97	5034650.88	5819700.18	48081.23	5625511.17
79	98	1691371.22	2007062.32	1100.55	2146322.66
80	99	11325.37	19987.67	NaN	26173.66

	Dept	Store	value
0	1	1	3219405.18
1	2	1	6592598.93
2	3	1	1880518.36
3	4	1	5285874.09
4	5	1	3468885.58
...	...	...	...
3640	95	45	7564151.83
3641	96	45	5.94
3642	97	45	924775.55
3643	98	45	75767.27
3644	99	45	NaN

Unit 7

Introduction To Numpy

What will you learn

- creating arrays with Numpy
- adding or subtracting arrays
- selecting elements from 1-D Array
- selecting elements from 2-D Array
- higher dimensional array

Numpy - Numerical Python

- NumPy, which stands for Numerical Python, has many uses including:
  - Working with many numbers at once
  - Generating random numbers
  - Performing different numerical functions, for example mean, median
  - It serves as the foundation for many other libraries and tools, such as pandas, SciPy, scikit-learn, and more.

Numpy Array

- NumPy includes a powerful data structure known as an **array**. It's a data structure that organizes multiple items. Each item can be of any type (strings, numbers, or even other arrays)
- Arrays are most powerful when they are used to store numbers. This is because arrays give us special ways of performing mathematical operations that are both simpler to write and more efficient computationally.

Example between list and array:

```
a = [1,2,3,4,5]
b = [1,2,3,4,5]

a+b
[1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
```

```
c = np.array([1,2,3,4,5])
d = np.array([1,2,3,4,5])

c+d
array([ 2, 4, 6, 8, 10])
```

Two-Dimensional Arrays

- We can create an array of arrays. If the arrays that make up our bigger array are all the same size, then it has a special name: a two-dimensional array.

1 D ARRAY:

C	O	D	I	N	G	E	E	K
0	1	2	3	4	5	6	7	8

single row of elements

2 D ARRAY:

		col 0	col 1	col 2
	i \ j	0	1	2
row 0	0	A	A	A
row 1	1	B	B	B
row 2	2	C	C	C

column

array elements

rows

Selecting Elements from a 1-D Array/2-D Array

- NumPy allows us to select elements from an array using their indices.
- This is identical to the native python list operations.

```
a = np.array([5, 2, 7, 0, 11])

a[0]   → 5
a[1:3] → array([2, 7])
a[:3]  → array([5, 2, 7])
a[-3:] → array([ 7, 0, 11])
```

- Selecting elements from a 2-d array is very similar to selecting them from a 1-d array, we just have two indices to select from. The syntax for selecting from a 2-d array is **array[row,column]**.

```
a = np.array([
[32, 15, 6, 9 , 14],
[12, 10, 5, 23, 1],
[2, 16, 13, 40, 37]
])
```

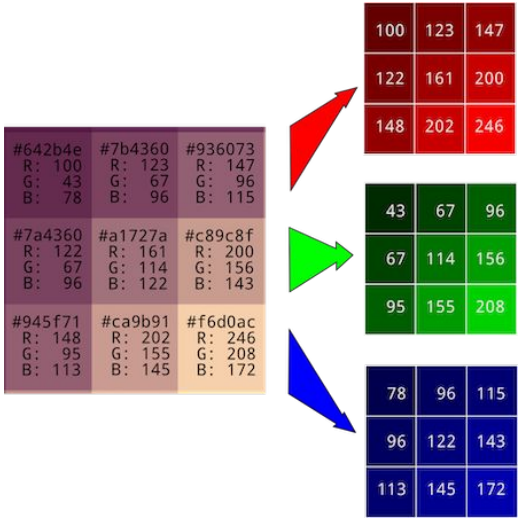
```
a[2,1]  → 16
a[:,0]  → array([32, 12, 2])
a[1]    → array([12, 10, 5, 23, 1])
a[0,:3] → array([32, 15, 6])
```

Higher dimension

- Arrays in NumPy are the data structures with high performance which are suitable for mathematical operations.
- The three levels of arrays nested inside one another represent the three-dimensional array in python, where each level represents one dimension.
- We make use of the array function in NumPy to create a three-dimensional array with an object as the parameter passed to it.

For example, an image is a 3-dimensional array.
•1st Dimension: Row
•2nd Dimension: Column
•3rd Dimension: Pixel

Pixels:
•[255, 255, 255] is a white pixel.
•[0, 0, 0] is a black pixel.
•[12, 32, 34] defines a pixel with 12/255 red, 32/255 green and 34/255 blue.



Unit 8

Statistics In NumPy

What will you learn

- different methods to calculate common statistical properties of a dataset
- understand how we can use statistics to summarize the data

Statistics in NumPy

- We can use Numpy to calculate common statistical properties of a dataset and summarize the data, for example:

Mean

```
survey_responses = [5, 10.2, 4,.3, 6.6]
survey_array =
np.array(survey_responses)
np.mean(survey_array)
5.220000000000001
```

Logical operations with Mean

```
np.mean(survey_array > 8)
0.2
```

The Mean of 2D Arrays

```
ring_toss = np.array([[1, 0, 0],
[0, 0, 1],
[1, 0, 1]])
np.mean(ring_toss)
0.4444444444444444

np.mean(ring_toss, axis=1)
array([0.33333333, 0.33333333, 0.66666667])
```

Sorting

- We can use **np.sort** to sort our data:

```
heights = np.array([49.7, 46.9, 62, 47.2, 47, 48.3, 48.7])
print(heights)
print(np.sort(heights))
```

```
[49.7 46.9 62. 47.2 47. 48.3 48.7]
[46.9 47. 47.2 48.3 48.7 49.7 62. ]
```

Other aggregation functions

- There are other aggregation functions for statistic usage, for example:

Median

```
a = np.array([1, 1, 2, 3, 4, 5, 5])
np.median(a)
□3.0
```

Percentiles

```
d = [1, 2, 3, 4, 4, 4, 6, 6, 7, 8, 8]
np.percentile(d, 40)
□4.0
```

Standard Deviation

```
nums = np.array([65, 36, 52, 91, 63, 79])
np.std(nums)
□17.716909687891082
```

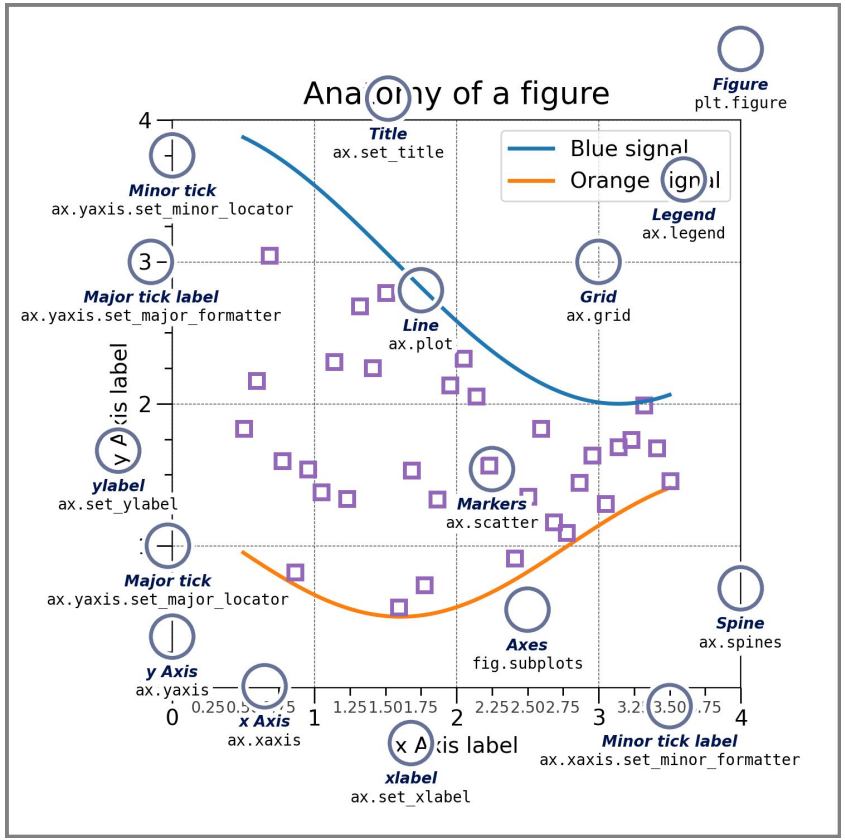
Unit 9

Matplotlib

What will you learn

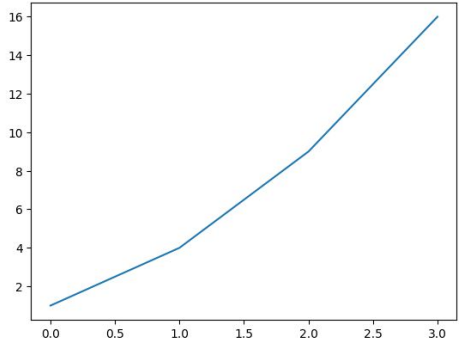
- concepts of a matplotlib graph
- line plot and bar plot
- scatter plot and histogram
- multiple plots in a graph and styling the plot

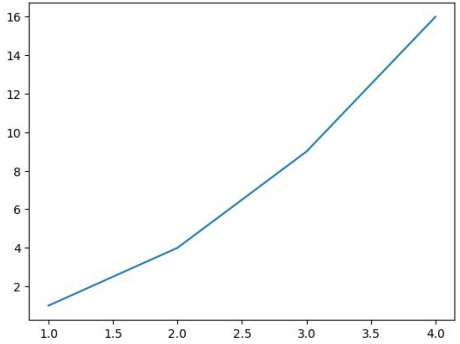
Concepts of a matplotlib graph

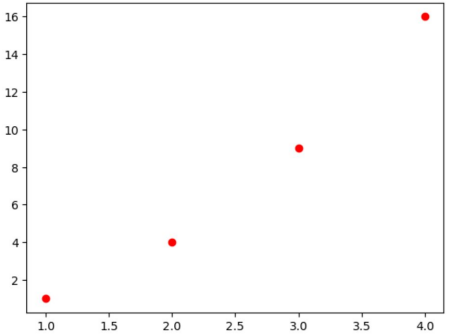


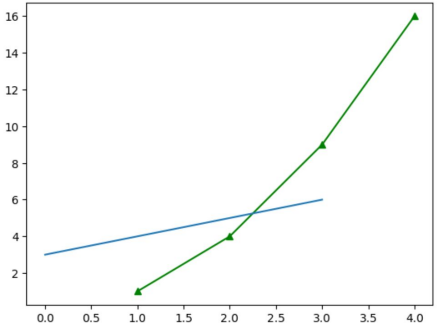
Line Plot

- plt.plot([1, 4, 9, 16]): the **single list** is treated as y-values, x-values are assumed to be range(len(first-list))
- plt.plot([1, 2, 3, 4], [1, 4, 9, 16]): the **first list** is treated as x-values, the **second list** is treated as y-values
- plt.plot([1, 2, 3, 4], [1, 4, 9, 16], "ro"): **Third parameter indicate the line style**
- plt.plot([1, 2, 3, 4], [1, 4, 9, 16], "g-^", [1, 4, 9, 16]): We can **add multiple lists in a graph**
- plt.plot("x", "y", "ro", data=<subscriptable>): In case we have a **pandas dataframe**, you want to plot with two columns







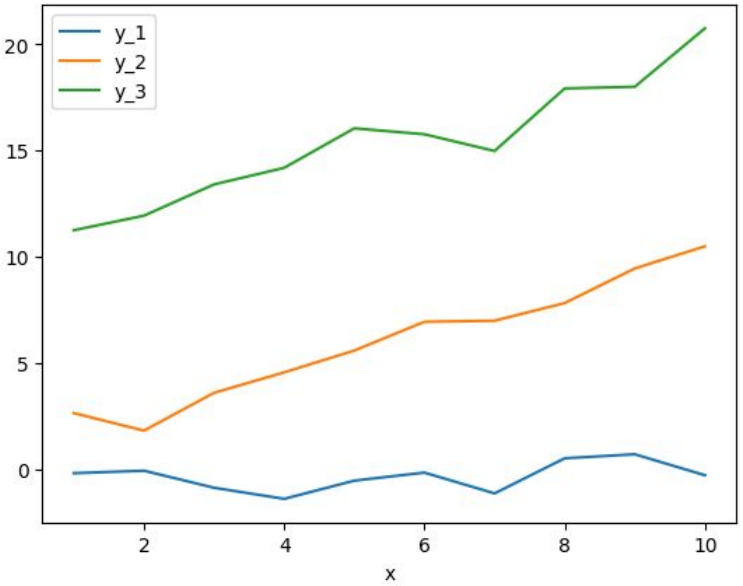


Pandas's Shortcut

- Pandas provides a very convenience shortcut to access matplotlib functionality. The syntax is much shorter:

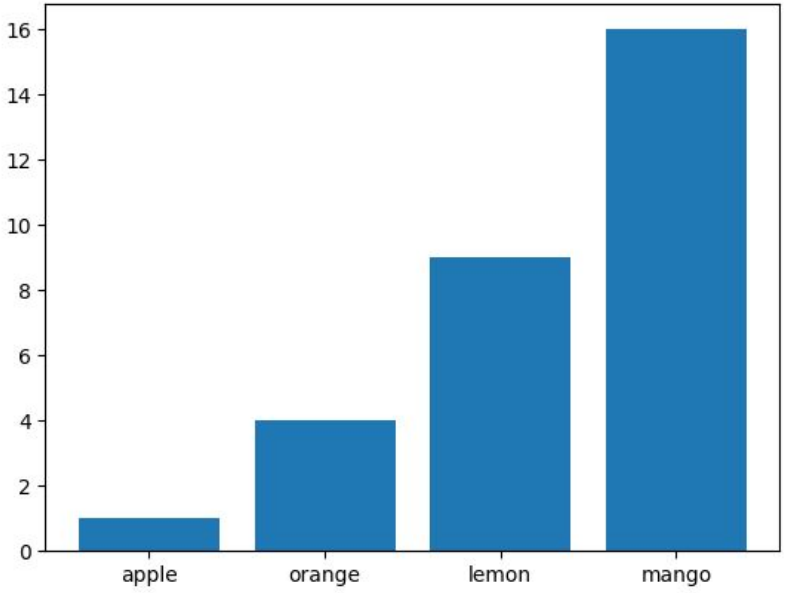
df.plot()

	x	y_1	y_2	y_3
0	1	-0.028984	1.709005	12.845869
1	2	0.468551	1.944360	11.925516
2	3	0.448893	0.635991	12.750074
3	4	0.220619	5.512786	14.426916
4	5	-1.320236	3.691454	13.973544



Bar plot on categorical data

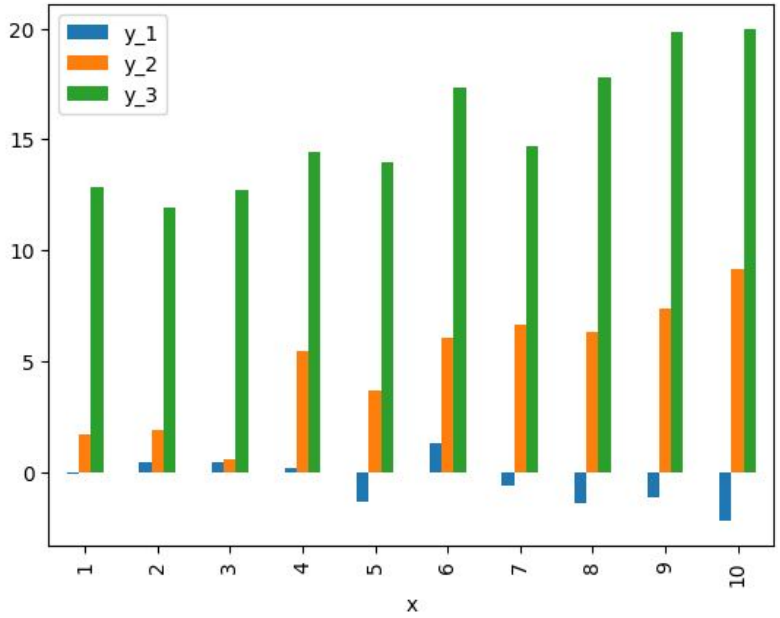
- You need both x and y values for a bar chart, for example:
  plt.bar(['apple', 'orange', 'lemon', 'mango'], [1, 4, 9, 16])



Pandas's Shortcut -- Barplot

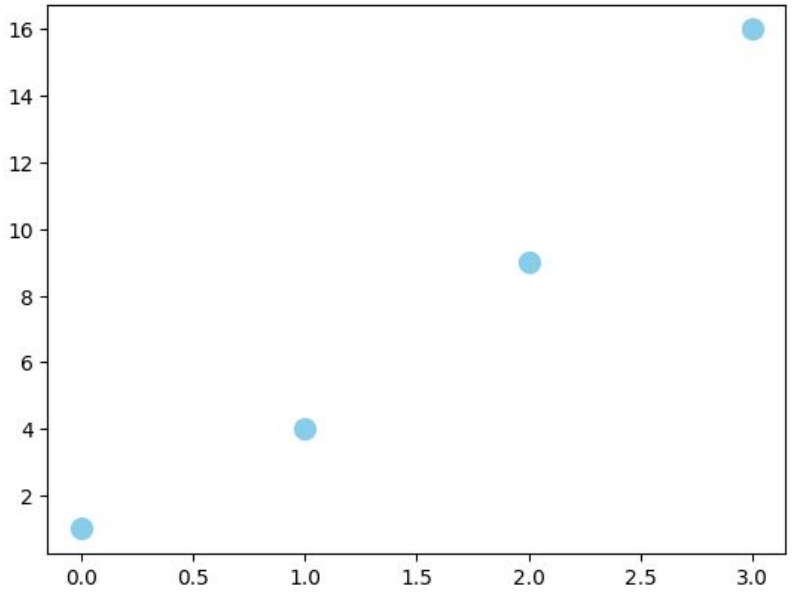
- Pandas provides a very convenience shortcut to access matplotlib functionality. The syntax is much shorter:

```
df.set_index("Col").plot.bar()
```



Scatter plot

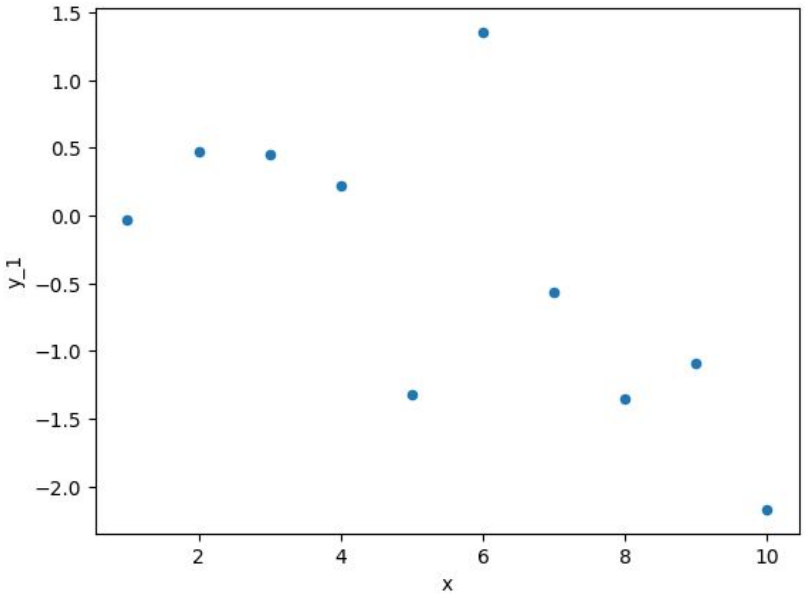
- First list will be set as x-value and second list will be set as y-value, for example:
  plt.scatter([0,1,2,3], [1, 4, 9, 16], c='skyblue', s=100)



Pandas's Shortcut -- Scatter

- Pandas provides a very convenience shortcut to access matplotlib functionality. The syntax is much shorter:

```
df.plot.scatter(x="x", y="y_1")
```



Histogram

- Histogram is the aggregated form of data. It groups a series of values into different bins of same size, e.g. 1-3, 3-5, 5-7
- We can also access it from the standard **plt.hist()** or pandas's **df.hist()**.

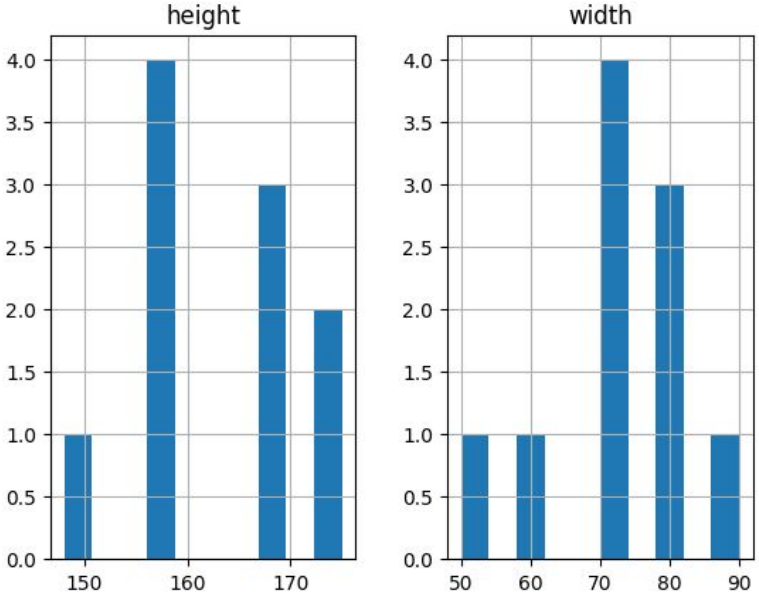
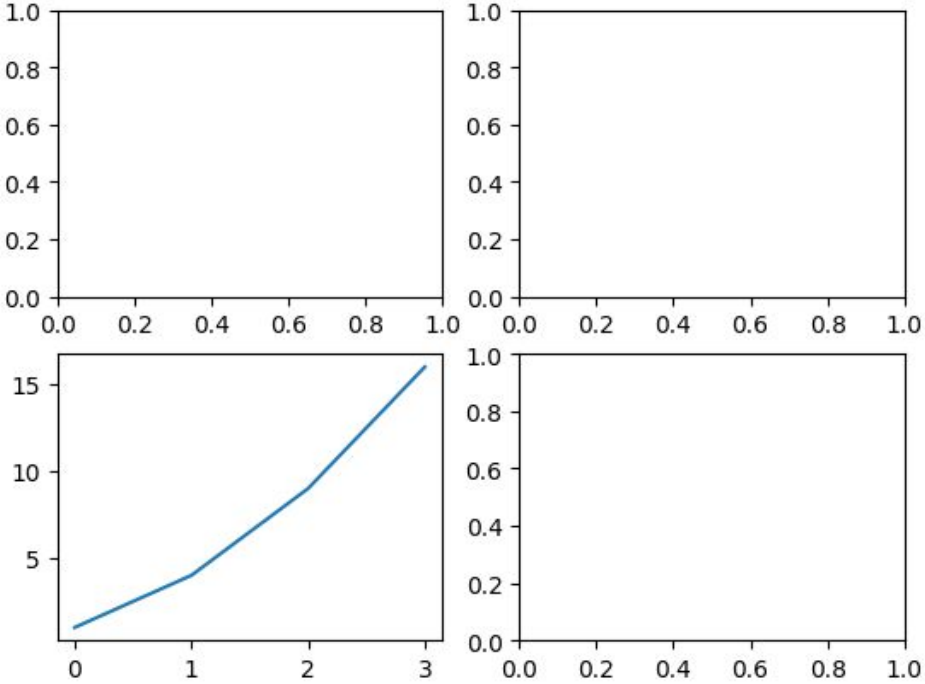


Figure and Axe

- When we do plt.xxx(), matplotlib plot it onto a predefined axe in a pre-defined fig.
- However, we can have control over the figure and the axes, for example:

```
fig, axs = plt.subplots(2, 2)
axs[1][0].plot([1, 4, 9, 16])
```

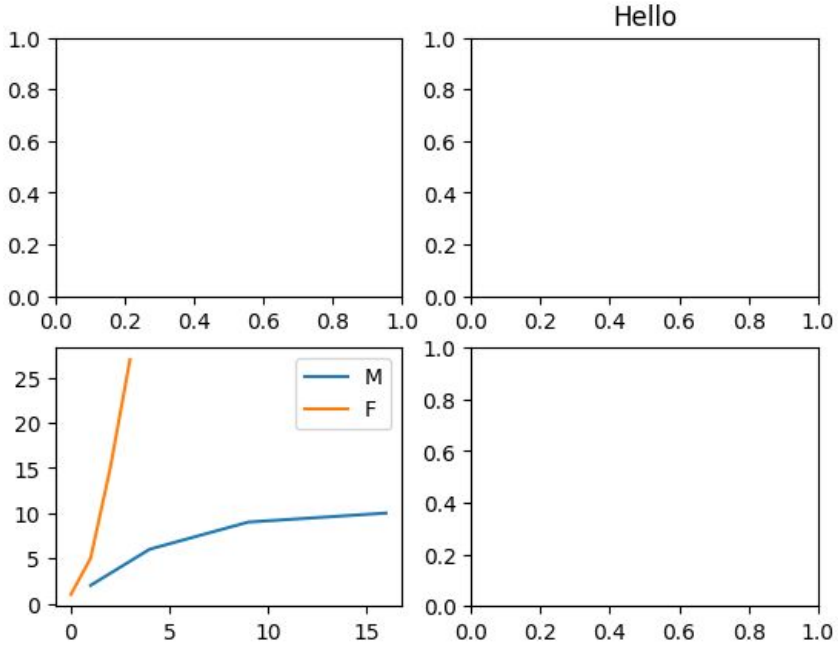


Styling your plot

- We can add styling to the plot, common command:
  - Add a title to your figure: fig.suptitle('')
  - Add a title to your axe: ax.set_title('')
  - Set x label: ax.set_xlabel('')
  - Set y label: ax.set_ylabel('')
  - Add legend: ax.legend()
  - Add text: ax.text()
  - Set some predefined style (This is a global setting): plt.style.use('seaborn-darkgrid') & plt.style.use('ggplot')
  - Set xtick: ax.set_xticks([])
  - Set xtick labels: ax.set_xticklabels([])
  - Set ytick: ax.set_yticks([])
  - Set ytick labels: ax.set_yticklabels([])

For example:

```
fig, axs = plt.subplots(2, 2)
axs[1][0].plot([1, 4, 9, 16], [2,6,9,10], [1,5,15,27])
axs[0][1].set_title('Hello')
axs[1][0].legend(['M','F'])
```



Divide into multiple series

- We can draw multiple plots onto the same axe. Also some style you may add.

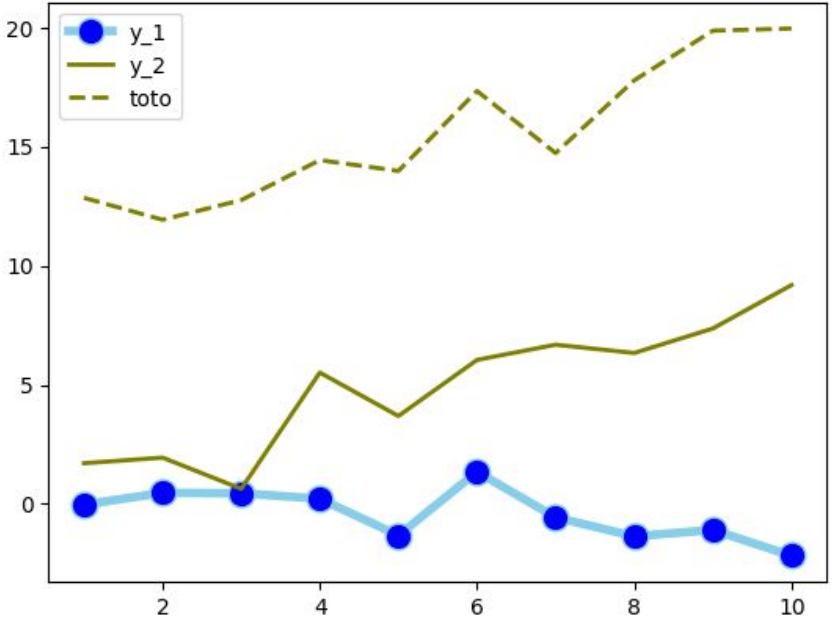
For example:

```
plt.plot( 'x', 'y_1', data=df, marker='o',
markerfacecolor='blue', markersize=12,
color='skyblue', linewidth=4)

plt.plot( 'x', 'y_2', data=df, marker='',
color='olive', linewidth=2)

plt.plot( 'x', 'y_3', data=df, marker='',
color='olive', linewidth=2, linestyle='dashed',
label="toto")

plt.legend()
```



Unit 10

Seaborn

What will you learn

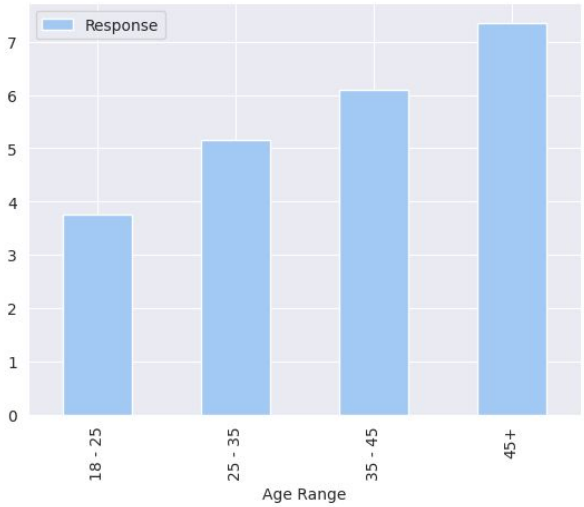
- plot graph with aggregated data
- group data by 2 dimensions with hue
- apply aggregate methods with estimator
- other useful plots such as replot(), boxplot() and heatmap()

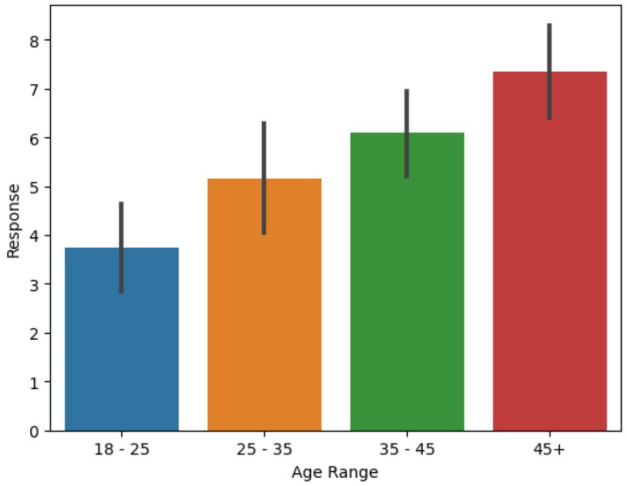
Introduction to Seaborn

- provides a more visually appealing plotting style and concise syntax.
- natively understands Pandas DataFrames.
- perform aggregations internally

For example:
In matplotlib: patient_df.groupby('Age Range')[['Response']].mean().plot.bar()
In seaborn: **sns.barplot**(x='Age Range', y='Response', data=patient_df)

	Patient ID	Gender	Age Range	Response
0	3951	Female	18 - 25	3
1	889	Male	18 - 25	5
2	2115	Female	18 - 25	4
3	3314	Female	18 - 25	2
4	5407	Male	18 - 25	6
...	...	...	...	...
75	6633	Female	45+	9
76	8337	Male	45+	9
77	6878	Female	45+	9
78	8721	Female	45+	7
79	1479	Female	45+	11

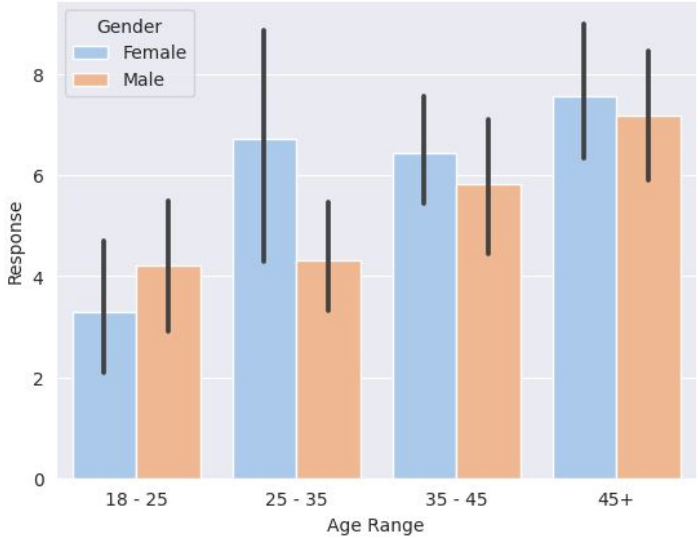




Group by 2 dimensions (hue)

- Sometimes we'll want to aggregate our data by multiple columns to visualize nested categorical variables.
- For example, consider our hospital survey data. The mean satisfaction seems to depend on Gender, but it might also depend on another column: Age Range.

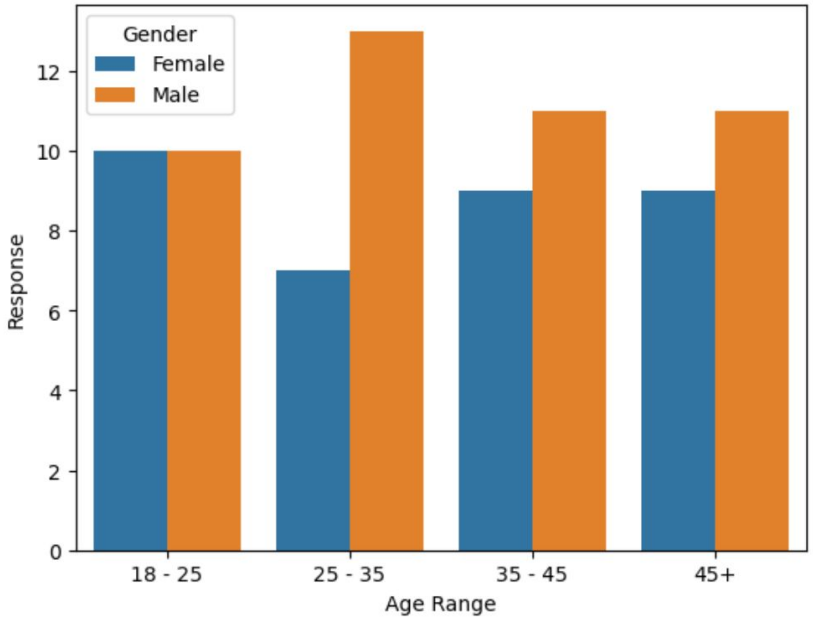
```
sns.barplot(x="Age Range", y="Response", data=patient_df, hue="Gender")
```



Aggregation methods (estimator)

- In most cases, we'll want to plot the mean of our data, but sometimes, we'll want something different.
- **sns.barplot**(... , **estimator**=afunc, ...)
- Consider our hospital satisfaction survey data, we'd like to know how many men and women answered the survey

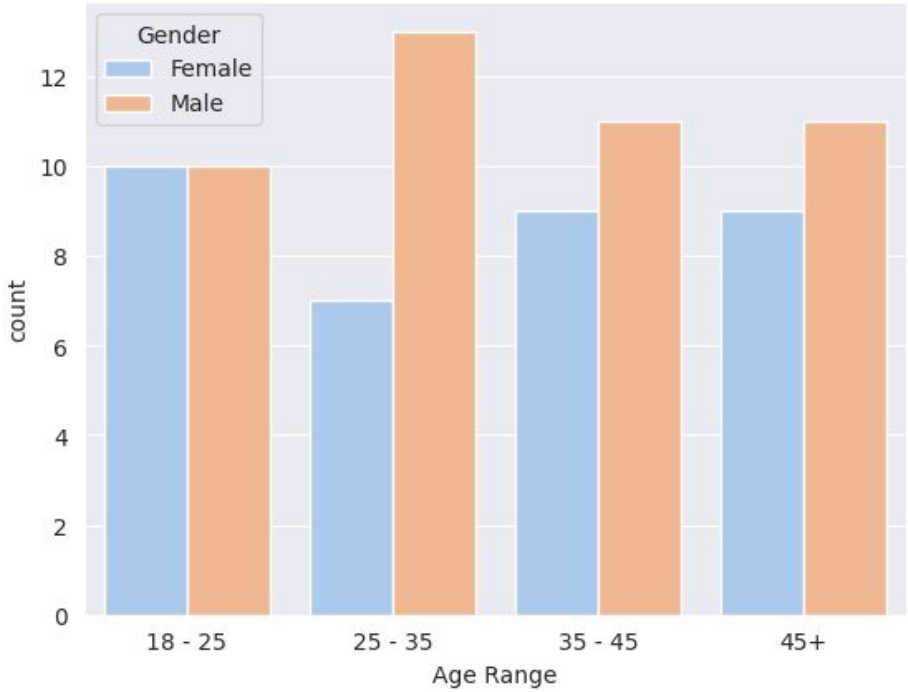
```
sns.barplot(x="Age Range", y="Response",
data=df, hue="Gender", estimator=len)
```



Countplot()

- The advantage of using countplot() is that it can work even if you do not have the Response column:

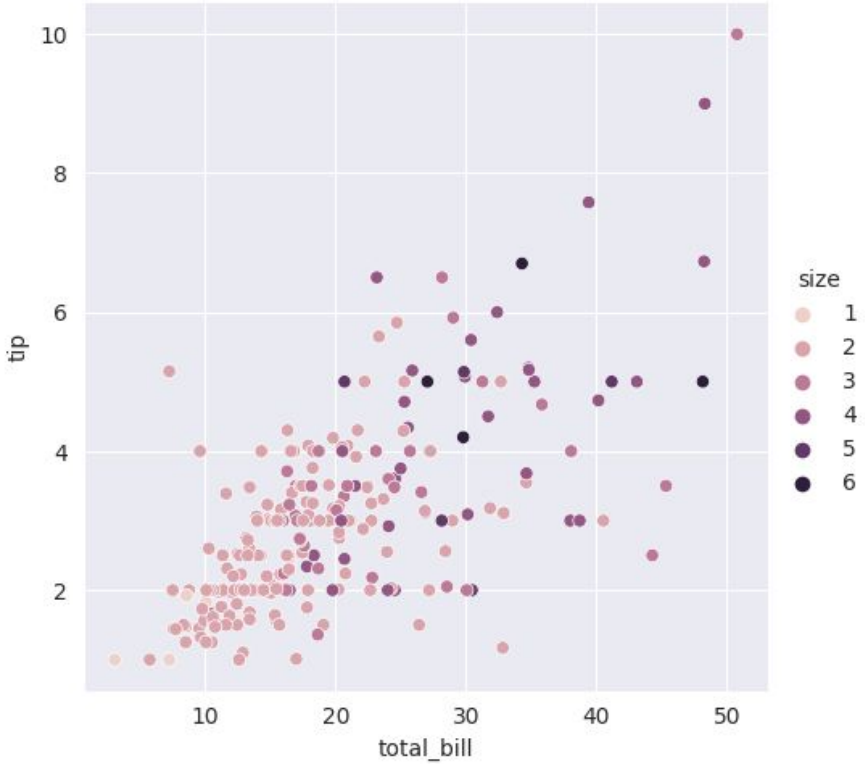
```
sns.countplot(data=patient_df,
hue="Gender", x="Age Range")
```



Relational Plots

- This is a figure-level function for visualizing statistical relationships using two common approaches: scatter plots and line plots.
- The scatterplot() is the default kind in relplot() (it can also be forced by setting kind="scatter"):

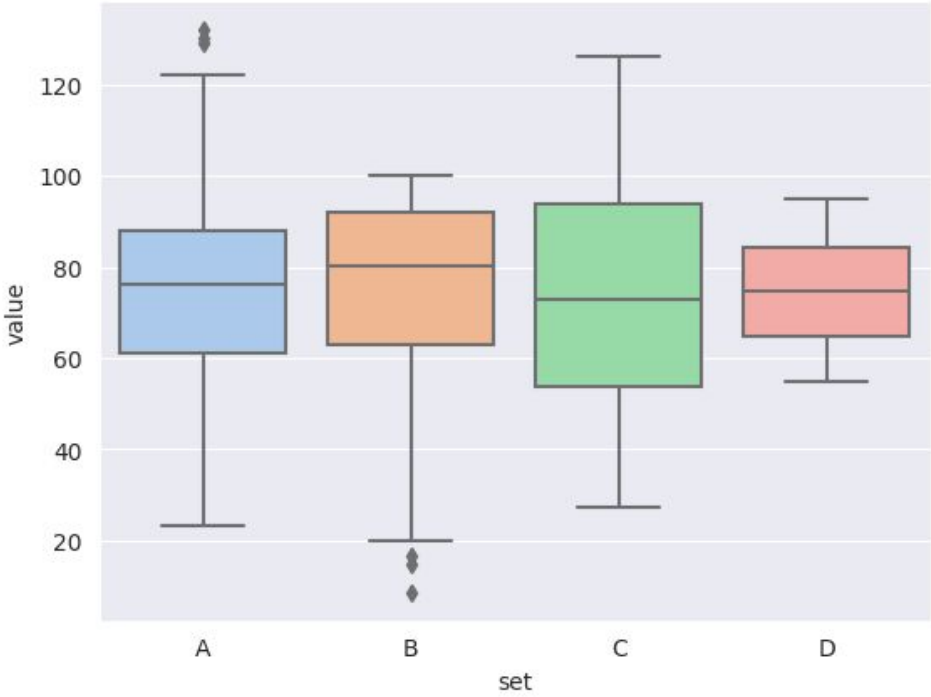
sns.relplot(data=tips, x='total_bill', y='tip', hue='size')



Box Plots

- The box plot shows us the range of our dataset, gives us an idea about where a significant portion of our data lies, and whether or not any outliers are present.
- We can read multiple information from the chart at one time:
  - median
  - lower quartile
  - upper quartile

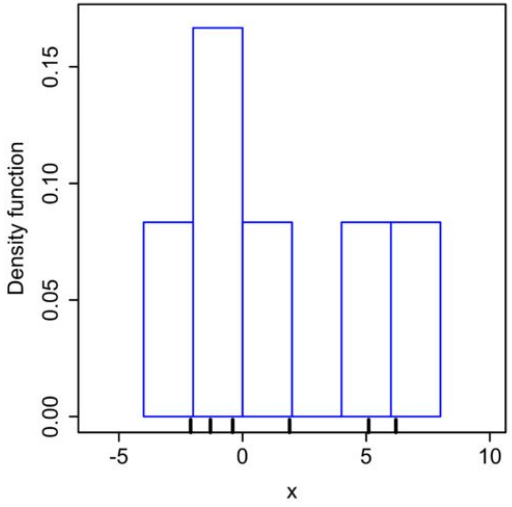
```
sns.boxplot(x='set',y='value',data=df3)
```

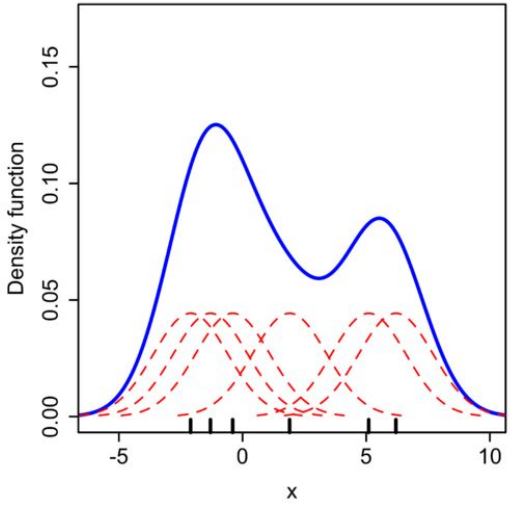


Histogram and KDE Plots

- A KDE plot gives us the sense of a univariate as a curve. A univariate dataset only has one variable and is also referred to as being one-dimensional

Histogram: sns.displot(df[Col])
KDE: sns.displot(df[Col], kde = True)

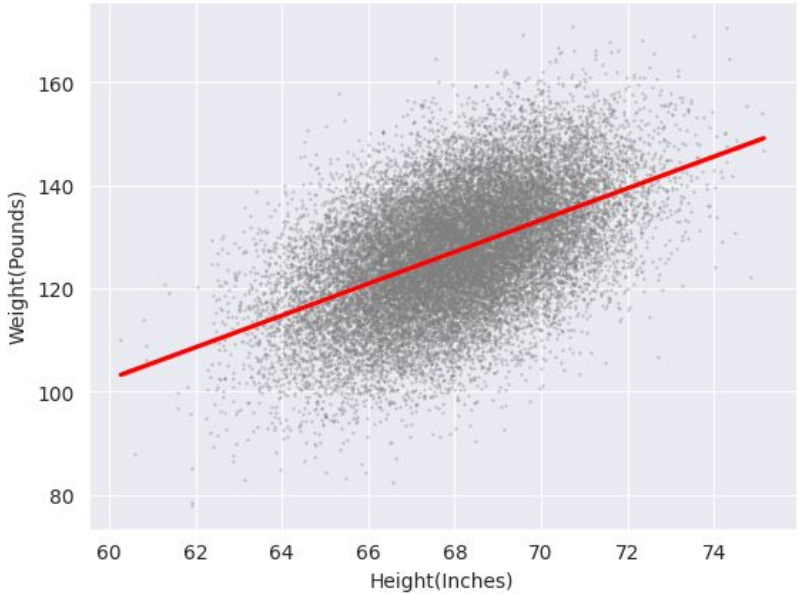




Reg Plot

- One of the handiest visualization tools for making quick inferences about relationships between variables is the scatter plot.

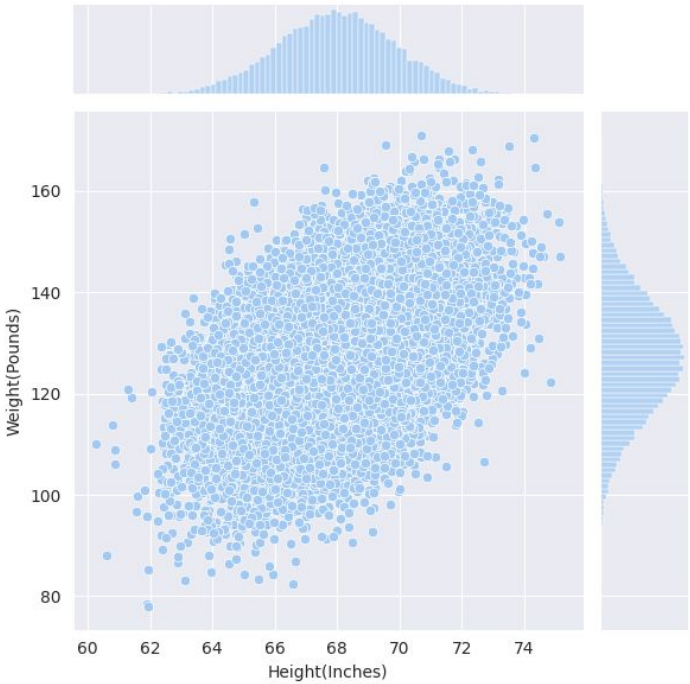
```
sns.regplot(x='Height(Inches)', y='Weight(Pounds)', data=df4,
line_kws={'color':'red'}, scatter_kws={'s':0.1,'color':'grey'})
```



Jointplot

- Jointplot is also a variable of scatter plot. It gives us also the individual distributions.

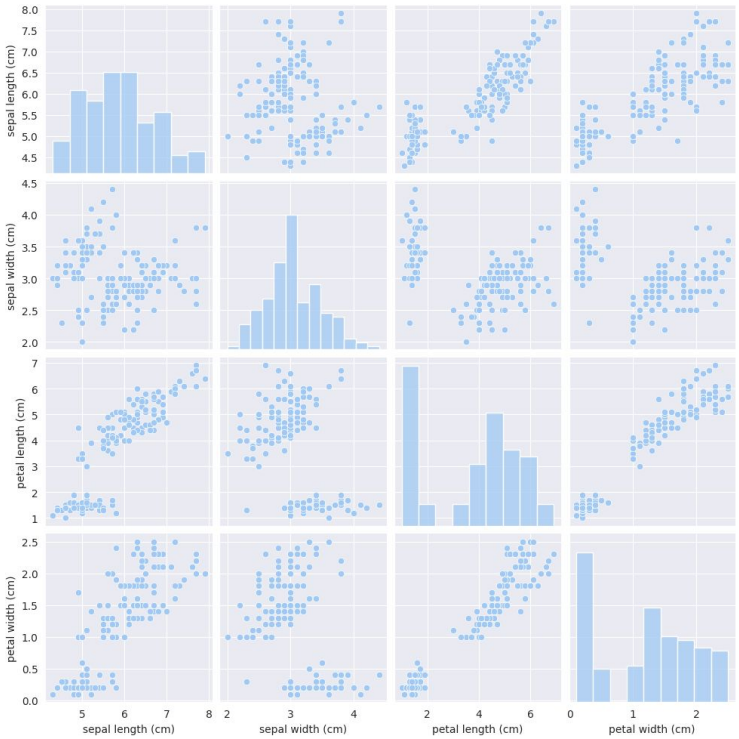
```
sns.jointplot(x='Height(Inches)', y='Weight(Pounds)', data=df4)
```



Pairplot

- Pairplot in seaborn create histogram or scatter plots for each pair of columns in the dataframe. It is very useful in data science when we want to find insight between features.

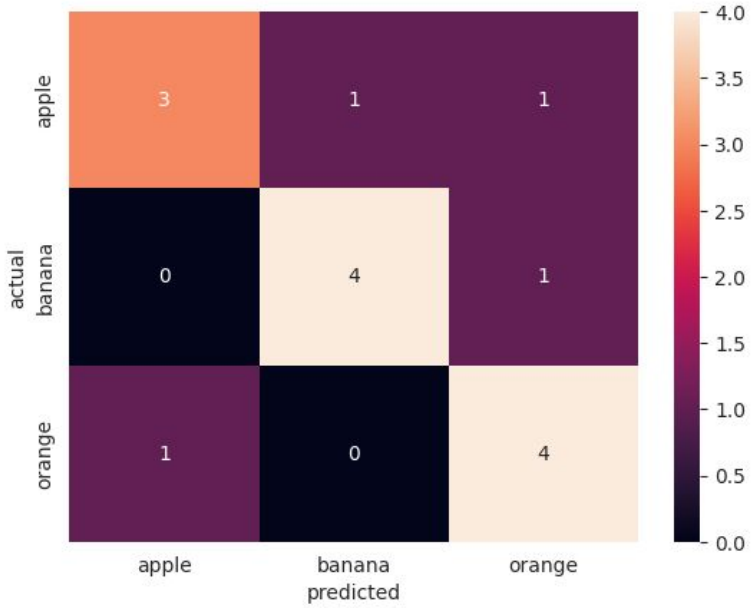
sns.pairplot(df)



Heatmap

- A heat map is a data visualization technique that shows magnitude of a phenomenon as color in two dimensions.
- The variation in color may be by hue or intensity, giving obvious visual cues to the reader about how the phenomenon is clustered or varies over space

```
pivoted =
observations.value_counts().unstack().fillna(0)
sns.heatmap(pivoted, annot=True)
```



Unit 11

Class

What will you learn

- create a class and instance
- define methods as part of a class
- use constructors to initialize the instance
- define how instances of a class behave with respect to various Python operations

Object-Oriented Programming

- A class instance is also called an object. The pattern of defining classes and creating objects to represent the responsibilities of a program is known as Object Oriented Programming (OOP)
- For example, Pandas is a very famous library in python. One its most used class is called DataFrame

Class & Instance

- A class is a template for a data type. It describes the kinds of information that class will hold and how a programmer will interact with that data
- For example, we created a class and named it Dog:
  - class Dog():
- Instantiation takes a class and turns it into an object, the type() function does the opposite of that
- When called with an object, it returns the class that the object is an instance
- For example, we created an instance named Bobby from the class Dog:
  - Bobby = Dog()

Methods

- Methods are functions that are defined as part of a class
- The first argument in a method is always the object that is calling the method. We usually name this first argument as self
- For example, we defined a method named sing_a_song() in the class Dog, with the first argument self:
  -
```
class Dog:
  def sing_a_song(self):
    print("woof-woof arf-arf ruff-ruff")
```
- To create an instance and call the method:
  -
```
Bobby = Dog()
Bobby.sing_a_song()
```

Constructors

- Methods that are used to prepare an object being instantiated are called constructors.
- The first dunder (Double Underscore) method we're going to use is the init method
- This method is used to initialize a newly created object. It is called every time the class is instantiated. For example:
  - ```
    class Shouter:
      def __init__(self):
        print("Hello!")
    shout1 = Shouter() # prints "Hello!" when we created the instance named shout1
    ```
- We can pass additional arguments into the methods, for example:
  - ```
    class Dog:
     def __init__(self, color):
      print(f"I am making a dog with {color} color")
    a_blue_dog = Dog("blue")
    ```

Class variable vs Instance variable

- Class variable is attached to class definition, it has only one copy over multiple instance of the same type of object.
- Instance variable is attached to instance, it has one copy per instance.
- For example:
  -
```
class Dog:
    number_of_legs = 4  # This is a class variable
    def __init__(self, c):
        self.color = c  # This is an instance variable
    def sing_a_song(self):
        print(f"woof-woof arf-arf ruff-ruff, i am a {self.color} dog.")
```

String Representation

- **__repr__** is a method we can use to tell Python what we want the string representation of the class to be
- repr can only have one parameter, self, and **must return a string**
- For example:

```
class Employee():
    def __init__(self, name):
        self.name = name
    def __repr__(self):
        return self.name
bruce = Employee("Bruce Lee")
print(bruce)
```

Magic methods

- Magic methods in Python are the special methods that start and end with the double underscores. It adds functionality to our custom classes, for example:
  - __repr__(self): return a machine readable representation of a type
  - __ge__(self, other): comparison using >= operator
  - __le__(self, other): comparison using <= operator